Playlist Manager API

A RESTful Web API for managing music playlists, built with **ASP.NET Core (.NET 10)** and **Entity Framework Core**. Users can create playlists, add songs to them, and retrieve their own playlists, with the database enforcing ownership, relationships, and data integrity.

Table of Contents

Overview

The Playlist Manager API is a backend service that lets a user maintain personal music playlists. It is organized in clean layers — **Controllers → Services → EF Core `DbContext` → Models** — with separate request and response DTOs so the public contract is decoupled from the database entities.

A **relational SQL database** was chosen deliberately. The domain is inherently relational (users own playlists, playlists contain songs), and a SQL engine gives three things this application relies on:

- **Structured relationships** between users, playlists, and songs.
- **Foreign keys** that guarantee a playlist always belongs to a real user and every join row references a real playlist and a real song.
- **Duplicate prevention**, enforced by a composite primary key on the join table so the same song cannot be added to the same playlist twice — at the database level, not just in application code.

The default and primary target is **SQL Server (Express)**, but the project can be pointed at another provider such as PostgreSQL (see Using a Different Database Provider).

Features

- **Create a playlist** owned by a specific user.
- **Add a song to a playlist**, with ownership checks, existence checks, and duplicate prevention.
- **Fetch all playlists for a user**, returning only that user's playlists.
- **Delete a playlist**, which automatically removes its song links via cascade delete.
- Supporting song operations (create, list, search by id / name / artist).
- Interactive API documentation served through **Scalar** in development.

Architecture

Layer	Responsibility
Controllers	Handle HTTP requests, model validation, and status codes.
Services (`IPlaylistService` , `ISongService`)	Business rules: ownership, duplicate handling, ordering.
`AppDbContext`	EF Core mapping, relationships, and cascade behavior.
Models	Database entities (`User` , `Playlist` , `Song` , `PlaylistSongs`).
DTOs	Request/response shapes with validation attributes.

Dependencies flow toward abstractions: controllers depend on service interfaces, which are registered for dependency injection in `Program.cs` . EF Core's `DbContext` / `DbSet` provides the repository and unit-of-work pattern, so no separate repository layer is required.

Data Model

Four tables make up the schema:

- **Users** — `Id` (PK), `Name`
- **Songs** — `Id` (PK), `Name` , `Artist` , `PublishDate`
- **Playlists** — `Id` (PK), `Name` , `CreationDate` , `UserId` (FK → Users)
- **PlaylistSongs** (join table) — `PlaylistId` (FK → Playlists), `SongId` (FK → Songs), `OrderInPlaylist`

Relationships and integrity rules:

- A **User** has many **Playlists**. Deleting a user cascades to their playlists.
- A **Playlist** has many songs through **PlaylistSongs**. Deleting a playlist cascades to its join rows.
- A **Song** can appear in many playlists through **PlaylistSongs**. Deleting a song cascades to its join rows.
- The composite primary key **(`PlaylistId` , `SongId`)** guarantees a song appears in a given playlist at most once. `OrderInPlaylist` records position but is **not** part of the key.

```
Users (1) ──< Playlists (1) ──< PlaylistSongs >── (1) Songs
```

API Endpoints

Base path: `/api`

Method	Route	Description
`POST`	`/api/playlist`	Create a playlist.
`POST`	`/api/playlist/addsong`	Add a song to a playlist.
`GET`	`/api/playlist/search/userId/{userId}`	Get all playlists for a user.
`GET`	`/api/playlist/search/playlistId/{id}`	Get a single playlist by id.
`GET`	`/api/playlist/search/name/{name}`	Search playlists by name.
`DELETE`	`/api/playlist?playlistId={id}&userId={id}`	Delete a playlist (cascades to its songs).
`GET`	`/api/song`	List all songs.
`POST`	`/api/song`	Create a song.
`GET`	`/api/song/search/id/{id}`	Get a song by id.
`GET`	`/api/song/search/name/{name}`	Search songs by name.
`GET`	`/api/song/search/artist/{artist}`	Search songs by artist.

Example — create a playlist

```
POST /api/playlist
Content-Type: application/json

{
  "name": "Road Trip",
  "userId": 1
}
```

Example — add a song to a playlist

```
POST /api/playlist/addsong
Content-Type: application/json

{
  "playlistId": 1,
  "songId": 1,
  "userId": 1
}
```

Technology Stack

- ASP.NET Core Web API on **.NET 10**
- Entity Framework Core 10 (SQL Server provider)
- Scalar for API reference UI
- xUnit for tests

The following NuGet packages are already referenced by the project; you do **not** install them manually (a `dotnet restore` pulls them in):

- `Microsoft.EntityFrameworkCore`
- `Microsoft.EntityFrameworkCore.SqlServer`
- `Microsoft.EntityFrameworkCore.Tools`
- `Microsoft.AspNetCore.OpenApi`
- `Scalar.AspNetCore`

> If a build error reports one of these as missing, run `dotnet restore` (or *Restore NuGet Packages* in Visual Studio) before reinstalling anything by hand.

Prerequisites

- **.NET 10 SDK** — required to build and run the project and to run EF Core migration commands. (Installing only the runtime is not enough.)
- **A SQL database.** SQL Server 2022/2025 **Express** is the primary target on Windows. On macOS or Linux, use SQL Server in Docker or switch to PostgreSQL — see Using a Different Database Provider.
- One of:
- **Visual Studio 2022/2026** (includes the Package Manager Console), or
- **VS Code / any editor + the `dotnet ef` CLI tool** (cross-platform).

Getting Started

1. Clone and restore

```
git clone <your-repo-url>
cd PlaylistManagerApi
dotnet restore
```

2. Configure the database connection

The connection string lives in `appsettings.json` under `ConnectionStrings:DefaultConnection`. You can change the database name there. The default (SQL Server Express) is:

```
"ConnectionStrings": {
  "DefaultConnection": "Server=localhost\\SQLExpress;Database=StorageDb;Trusted_Connection=True;TrustServerCertificate=True"
}
```

> Keep `Trusted_Connection=True;TrustServerCertificate=True` for a standard local Express install — they enable Windows authentication and accept the local self-signed certificate. Without them the connection usually fails.

3. Apply the database migration

The project ships with an existing `Initial` migration. You only need to apply it.

Option A — `dotnet ef` CLI (cross-platform, recommended):

```
# Install the EF tool once (global):
dotnet tool install --global dotnet-ef

# From the folder that contains PlaylistManagerApi.csproj:
dotnet ef database update
```

Option B — Visual Studio Package Manager Console (Windows):

Open **View → Other Windows → Package Manager Console**, set the *Default project* dropdown to **PlaylistManagerApi**, then run:

```
Update-Database
```

Clean re-create (only if you want to rebuild the schema from scratch):

```
# CLI
dotnet ef database drop
dotnet ef database update
```

```
# Package Manager Console
Drop-Database
Update-Database
```

> **Do not** run `Add-Migration Initial` on the unchanged project — a migration named `Initial` already exists and the command will fail. Only add a new migration after you change the model, and give it a new name (for example `Add-Migration AddPlaylistDescription`).

4. Seed sample data (optional)

Creating a playlist requires an existing **User**, and there is no user-creation endpoint, so seed at least one user before testing. Run the SQL below in **SQL Server Management Studio (SSMS)** or the Visual Studio SQL Server Object Explorer (right-click the database → **New Query**), or place it in a `data_insertions.sql` file in the repo:

```
INSERT INTO Users (Name) VALUES ('Asmaa'), ('Sara');

INSERT INTO Songs (Name, Artist, PublishDate) VALUES
  ('Bohemian Rhapsody', 'Queen', '1975-10-31'),
  ('Imagine', 'John Lennon', '1971-09-09');

INSERT INTO Playlists (Name, CreationDate, UserId) VALUES
  ('Road Trip', GETUTCDATE(), 1);

-- Link the first song to the first playlist at position 1.
INSERT INTO PlaylistSongs (PlaylistId, SongId, OrderInPlaylist) VALUES (1, 1, 1);
```

> Do not supply explicit values for identity `Id` columns; the database assigns them. Insert in dependency order (Users and Songs first, then Playlists, then PlaylistSongs).

5. Run the API

```
dotnet run --project PlaylistManagerApi
```

Or press **Run** (the `https` profile) in Visual Studio.

On first HTTPS run you may be prompted to trust the local development certificate:

- **Visual Studio:** when asked to trust the IIS Express / ASP.NET Core SSL certificate, click **Yes**.
- **CLI / cross-platform:** run `dotnet dev-certs https --trust` once.

With the app running in development, open the interactive docs at:

```
https://localhost:7230/scalar
```

You can also exercise every endpoint using the included `PlaylistManagerApi.http` file (in Visual Studio or the VS Code REST Client). The default ports are `7230` (HTTPS) and `5239` (HTTP); if your launch profile differs, check the console output for the actual URL.

Using a Different Database Provider

The connection string and the provider call in `Program.cs` must match.

SQL Server (default):

```
// appsettings.json
"DefaultConnection": "Server=localhost\\SQLExpress;Database=StorageDb;Trusted_Connection=True;TrustServerCertificate=True"
```

```
// Program.cs
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

PostgreSQL:

```
// appsettings.json
"DefaultConnection": "Host=localhost;Port=5432;Database=StorageDb;Username=postgres;Password=yourpassword"
```

```
// Program.cs
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseNpgsql(builder.Configuration.GetConnectionString("DefaultConnection")));
```

Two extra steps are required to switch to PostgreSQL: 1. Install the provider package: `dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL`. (`UseNpgsql` does not exist until this package is added.) 2. The existing migration was generated for SQL Server (`nvarchar`, identity annotations, `datetime2`) and will not run on PostgreSQL. Delete the `Migrations` folder, then regenerate: `dotnet ef migrations add Initial` followed by `dotnet ef database update`.

Running the Tests

The solution includes an xUnit test project covering create / add-song / fetch logic, validation, ownership, cascade deletes, and foreign-key enforcement.

```
dotnet test
```

Troubleshooting

Symptom	Cause / Fix
Build error: a NuGet package is "missing"	Run `dotnet restore` (or restore packages in Visual Studio).
`dotnet ef` not recognized	Install the tool: `dotnet tool install --global dotnet-ef`, then reopen the terminal.
`Remove-Migration` / `Add-Migration Initial` fails	An `Initial` migration already exists and/or has been applied. Use `Update-Database` to apply it, or `Drop-Database` then `Update-Database` to rebuild.
Cannot connect to SQL Server	Confirm SQL Express is running and the instance name matches (`localhost\SQLExpress`). Keep `Trusted_Connection=True;TrustServerCertificate=True`.
Creating a playlist returns "User not found"	Seed a user first (see Seed sample data); `UserId` must reference an existing user.
Browser shows a certificate warning	Run `dotnet dev-certs https --trust` (CLI) or accept the prompt in Visual Studio.
`/scalar` returns 404	Scalar is mapped only in the Development environment. Ensure `ASPNETCORE_ENVIRONMENT=Development`.

Project Structure

```
PlaylistManagerApi/
├── Controllers/          # PlaylistController, SongController
├── Services/             # IPlaylistService, PlaylistService, ISongService, SongService
├── Data/                 # AppDbContext (relationships + cascade config)
├── Models/               # User, Playlist, Song, PlaylistSongs
├── Dtos/                 # Request/response DTOs with validation
├── Migrations/           # EF Core migration + model snapshot
├── appsettings.json      # Connection string
├── Program.cs            # DI registration and pipeline
└── PlaylistManagerApi.http   # Sample requests for every endpoint

PlaylistManagerApi.Tests/ # xUnit tests (service, controller, relational)
```